МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Объектно-ориентированное программирование»
Тема: «Создания игрового приложения. Часть вторая»

Студент гр. 4388 _____ Кузьмин Г.И.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2025

Цель работы.

Расширение базовой архитектуры игрового приложения с использованием принципов объектно-ориентированного программирования: наследования, полиморфизма и работы с абстрактными интерфейсами. Реализация системы заклинаний и новых типов игровых объектов.

Задание.

1. Создать интерфейс карточки заклинания. Заклинание должно применяться игроком. На использование заклинания игрок тратит один ход.
2. Создать класс “руки” игрока, которая содержит все карточки заклинаний, которые игрок может применить в свой ход. Изначально рука игрока содержит только одно случайное заклинание. Реализовать возможность получать новые заклинание игроком, например, тратить очки на покупку или после уничтожения определенного кол-ва врагов. Размер “руки” должен быть ограничен и задается через конструктор.
3. Реализовать интерфейс заклинанием прямого урона. Это заклинание при использовании должно наносить урон врагу или вражескому зданию, если они находятся в достижимом радиусе. Если в качестве цели не выбран враг или вражеское здание, то заклинание не используется.
4. Реализовать интерфейс заклинания урона по площади. Это заклинание при использовании в допустимом радиусе наносит урон по области 2 на 2 клетки. Заклинание используется, даже если там нет никого.
5. Реализовать интерфейс заклинания ловушки. Заклинание размещает на поле ловушку, если враг наступает на клетку с ловушкой, то ему наносится урон, и ловушка пропадает.
6. Создать класс вражеской башни. Вражеская башня размещается на поле, и если в радиусе ее атаки появляется игрок, то применяет ослабленную версию заклинания прямого урона. Не может применять заклинание несколько ходов подряд.

Примечание

- Интерфейс заклинания должен быть унифицирован, чтобы их можно было единообразно использовать через интерфейс. Не должно быть методов в интерфейсе, которые не используются каким-то классом наследником.
- Избегайте явных проверок на тип данных.

Выполнение работы.

В данной лабораторной работе были реализованы новые классы и переработаны уже имеющиеся. Был добавлен класс интерфейса для заклинаний. Переделана визуальная часть приложения. Ниже приведено описание новых классов и существенных изменений уже написанных.

- **Интерфейс ISpell** – абстрактный базовый класс, определяющий единое взаимодействие для всех заклинаний в игре. Он не содержит полей, что гарантирует легкость архитектуры и отсутствие лишних зависимостей. От него наследуются все возможные типы заклинаний.

Методы класса:

1. virtual ~ISpell() = default – виртуальный деструктор для корректного удаления объектов наследуемых классов
2. apply(targetX, targetY, field, enemies) — виртуальный метод, реализующий логику активации магии.
3. getName() — возвращает строковое имя заклинания для отображения при игре.

- **Класс DirectDamageSpell** – класс, реализующий заклинание прямого урона. Наследуется от ISpell. При использовании наносит урон врагу, если он находится в достижимом радиусе. Если в качестве цели выбраны не враги, заклинание не используется

Методы класса:

1. bool apply(int targetX, int targetY, GameField& field, std::vector<Enemy>& enemies) override – метод, реализующий логику применения заклинания. Проверяет наличие врагов в радиусе действия. Устанавливает урон для врагов. Если врагов нет, заклинание не применяется.

2. std::string getName() const override – метод, возвращающий строку названия заклинания. Название – «Fireball (Direct)».

- **Класс AreaDamageSpell** – класс, реализующий заклинание урона по площади. Наследуется от ISpell. При использовании наносит урон врагу, если он находится в достижимом радиусе 2x2 клеток. Заклинание используется в любом случае.

Методы класса:

1. bool apply(int targetX, int targetY, GameField& field, std::vector<Enemy>& enemies) override – метод, реализующий логику применения заклинания. Проверяет наличие врагов в радиусе действия. Устанавливает урон для врагов. Применяется в любом случае.
2. std::string getName() const override – метод, возвращающий строку названия заклинания. Название – «Blizzard (Area 2x2)».

- **Класс TrapSpell** – класс, реализующий заклинание ловушки. Наследуется от ISpell. При использовании ставится на карту до активации врагом. Наносит ему урон. В случае срабатывания ловушка пропадает.

Методы класса:

1. bool apply(int targetX, int targetY, GameField& field, std::vector<Enemy>& enemies) override – метод, реализующий логику применения заклинания. Проверяет тип клетки и устанавливает на нее ловушку. Применяется при наступании на нее.
2. std::string getName() const override – метод, возвращающий строку названия заклинания. Название – «Spike Trap».

- **Класс Hand** – класс-контейнер, управляющий коллекцией заклинаний, доступных игроку. Использует std::vector с умными указателями std::unique_ptr, что обеспечивает автоматическое управление памятью и предотвращает утечки при удалении использованных карт.

Поля класса:

1. std::vector<std::unique_ptr<ISpell>> spells – вектор уникальных указателей на заклинания, доступных игроку. В случае лабораторной

ограничивается тремя единицами. Используется для хранения и реализации заклинаний.

2. size_t maxSize – максимальное количество заклинаний в руке

Методы класса:

1. Hand(size_t size) – конструктор, инициализирующий руку игрока с заданным максимальным количеством заклинаний.
2. bool addSpell(std::unique_ptr<ISpell> spell) – метод, который добавляет заклинание, если для него есть место.
3. void castSpell(size_t index, int x, int y, GameField& field, std::vector<Enemy>& enemies) – метод, который вызывается при разыгрывании заклинания.
4. const std::vector<std::unique_ptr<ISpell>>& getSpells() const { return spells; } – метод, который предоставляет доступ для отрисовки заклинаний в игре.

- **Класс EnemyTower** – класс, реализующий объект башни, предназначенная для охраны определенной области. В отличие от обычных врагов, башня не перемещается, но обладает механиками дистанционной атаки и перезарядки.

Поля класса:

1. int x, y – координаты башни.
2. int damage – урон, наносимый башней.
3. int range – радиус действия башни.
4. int cooldown – время перезарядки в ходах игрока.
5. int currentCooldown – текущее значения времени перезарядки.

Методы класса:

1. EnemyTower(int x, int y, int dmg, int rng, int cd) – конструктор, устанавливающий координаты, радиус атаки, урон и время перезарядки.
2. int update(Player& player) – метод для обновления времени перезарядки каждый ход и попытки атаки. Если игрок находится рядом с башней, то происходит атака. Возвращает количество нанесенного урона.

3. int getX() const { return x; } – возвращает координату X башни.
4. int getY() const { return y; } – возвращает координату Y башни.

Далее приведены классы, которые были переработаны.

- **Класс Game** – центральный класс, который выстраивает взаимодействие всех новых систем. Он связывает систему очков игрока с возможностью покупки новых заклинаний и управляет жизненным циклом ловушек на поле.

  Поля класса, которые были добавлены:

  1. std::vector<std::string> battleLog – вектор для записи логов событий во время игры.
  2. std::vector<EnemyTower> towers – вектор, который хранит список башень на игровом поле.

  Методы класса, которые были модифицированы:

  1. void render() – метод модифицирован для отображения сетки, полосок здоровья врагов и списка карт
  2. void handleInput() – в метод была добавлена обработка клавиш выбора заклинаний и их покупка.
  3. void update() – метод дополнен логикой срабатывания ловушек при попадании на них.

- Класс Player – основной класс для игрока.

  Поля класса, которые были добавлены:

  1. Hand hand – поле объекта заклинания. Позволяет игроку владеть рукой.
  2. Hand& getHand() – метод, реализующий доступ к руке игрока.
  3. void addScore(int points) – метод модифицирован, дабы позволить вычитать очки при покупке заклинания.

Далее приведена UML – диаграмма классов для данной лабораторной работы, отражающая связи между классами.

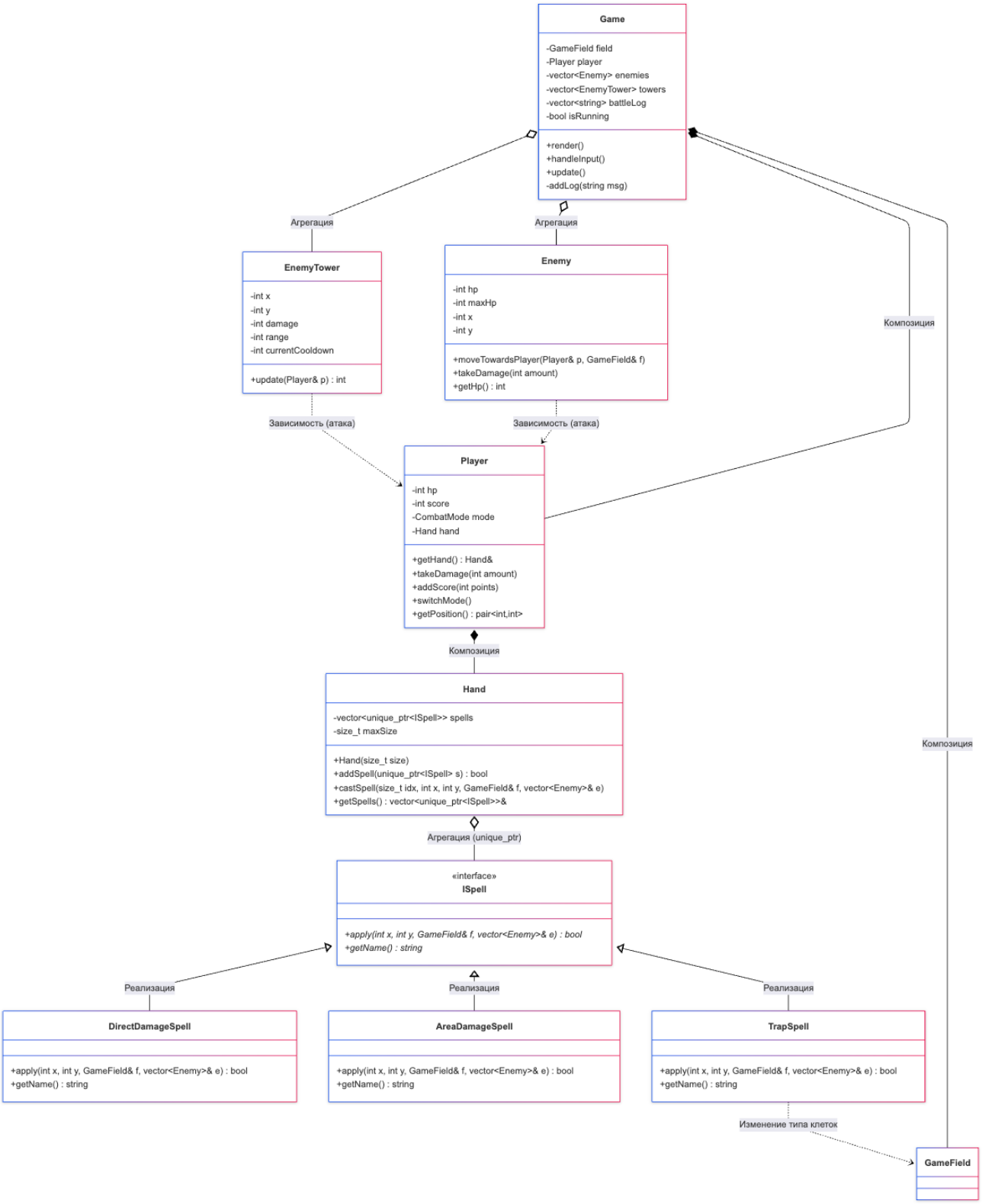

Рис 1. – UML-диаграмма классов

Тестирование.

1. Состояние игры сразу после запуска приложения.

```
 [ PLAYER ] HP: 100/100 | Score: 0 | Mode: MELEE
 [ ENEMIES ]
  E1: 30/30 HP [==========]
  E2: 30/30 HP [==========]
----------------------------------------
. . . . . # # . . . # . . . .
. P . . . . . . . . . . . . .
. . . # . # . . . . . . . . .
. # # . . . . . . . . . # # .
. . # . . . . . . . . . . . #
. . . # # E1. . . . . . . . .
. . # . . . . . . . . . . . .
# . . . . # . . . . . . . # .
. . . . . . # # T # . # . . #
. . . . . # . . . . . . . . .
# . . . . . . . . . . . . . #
. . . # . . . . . . # . . . .
. . . . . . . . . . . E2. #
. . . . . . . . # . . . . . .
. . . . . . . # . . . # . . .
---------- [ BATTLE LOG ] ----------
 > Waiting for action...
------------------------------------
 Spells: [1:Fireball (Direct)]
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис 2. – Запуск приложения

2. Проверка заклинания прямого урона и получение очков за убийство врага.

```
 Spells: [1:Fireball (Direct)]
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command: 1
Direction (WASD) for Fireball (Direct): s
 [ PLAYER ] HP: 90/100 | Score: 100 | Mode: MELEE
 [ ENEMIES ]
  E1: 30/30 HP [==========]
----------------------------------------
. . . . . # # . . . # . . . .
. . . . . . . . . . . . . . .
. . . # . # . . . . . . . . .
. # # . . P . . . . . . # # .
. . # . . . . . . . . . . . #
. . . # # . . . . . . . . . .
. . # . . . . . . . . . . . .
# . . . . # . . . . . . . # .
. . . . . . # # T # . # . . #
. . . . . # . . . . . . . . .
# . . . . E1. . . . . . . . #
. . . # . . . . . . # . . . .
. . . . . . . . . . . . . . #
. . . . . . . . # . . . . . .
. . . . . . . # . . . # . . .
---------- [ BATTLE LOG ] ----------
 > Casted Fireball (Direct)
 > Enemy defeated! +100 Score
------------------------------------
 Spells:
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис 3. – Заклинание прямого действия

3. Покупка случайного заклинания.

```
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command: b
 [ PLAYER ] HP: 90/100 | Score: 50 | Mode: MELEE
 [ ENEMIES ]
  E1: 30/30 HP [==========]
----------------------------------------
. . . . . # # . . . # . . . .
. . . . . . . . . . . . . . .
. . . # . # . . . . . . . . .
. # # . . P . . . . . . # # .
. . # . . . . . . . . . . . #
. . . # # . . . . . . . . . .
. . # . . . . . . . . . . . .
# . . . . # . . . . . . . # .
. . . . . . # # T # . # . . #
. . . . . # . . . . . . . . .
# . . . . E1. . . . . . . . #
. . . # . . . . . . # . . . .
. . . . . . . . . . . . . . #
. . . . . . . . # . . . . . .
. . . . . . . # . . . # . . .
---------- [ BATTLE LOG ] ----------
 > Casted Fireball (Direct)
 > Enemy defeated! +100 Score
 > Bought a new spell for 50 pts!
------------------------------------
 Spells: [1:Spike Trap]
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис. 4 – Покупка заклинания

4. Установка заклинания ловушки.

```
Enter command: 1
Direction (WASD) for Spike Trap: s
 [ PLAYER ] HP: 90/100 | Score: 50 | Mode: MELEE
 [ ENEMIES ]
  E1: 30/30 HP [==========]
----------------------------------------
. . . . . # # . . . # . . . .
. . . . . . . . . . . . . . .
. . . # . # . . . . . . . . .
. # # . . . . . . . . . # # .
. . # . . . . . . . . . . . #
. P . # # . . . . . . . . . .
. ^ # . . . . . . . . . . . .
# E1. . . # . . . . . . . # .
. . . . . . # # T # . # . . #
. . . . . # . . . . . . . . .
# . . . . . . . . . . . . . #
. . . # . . . . . . # . . . .
. . . . . . . . . . . . . . #
. . . . . . . . # . . . . . .
. . . . . . . # . . . # . . .
---------- [ BATTLE LOG ] ----------
 > Casted Fireball (Direct)
 > Enemy defeated! +100 Score
 > Bought a new spell for 50 pts!
 > Casted Spike Trap
------------------------------------
 Spells:
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис. 5 – Заклинание ловушки

5. Срабатывание ловушки.

```
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
 Enter command: w
  [ PLAYER ] HP: 90/100 | Score: 150 | Mode: MELEE
  [ ENEMIES ]
   All enemies defeated!
 ---------------------------------------
 . . . . . # # . . . # . . . .
 . . . . . . . . . . . . . . .
 . . . # . # . . . . . . . . .
 . # # . . . . . . . . . # # .
 . P # . . . . . . . . . . . #
 . . . # # . . . . . . . . . .
 . . # . . . . . . . . . . . .
 # . . . . # . . . . . . . # .
 . . . . . . # # T # . # . . #
 . . . . . # . . . . . . . . .
 # . . . . . . . . . . . . . #
 . . . # . . . . . . # . . . .
 . . . . . . . . . . . . . . #
 . . . . . . . . # . . . . . .
 . . . . . . . # . . . # . . .
 ---------- [ BATTLE LOG ] ----------
  > Enemy defeated! +100 Score
  > Bought a new spell for 50 pts!
  > Casted Spike Trap
  > E step on TRAP! -40 HP
  > Enemy defeated! +100 Score
 ----------------------------------
  Spells:
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
 Enter command:
```

Рис 6. – Активация ловушки

6. Проверка переполнения руки.

```
Enter command: b
 [ PLAYER ] HP: 30/100 | Score: 100 | Mode: MELEE
 [ ENEMIES ]
  All enemies defeated!
---------------------------------------
. . . . . . . . . . . . . # # .
. . . . . . . # . . . . . # .
. . . . . . . . . . # . . . .
. . . . . # . # . . . # . . .
. . . # . . . . # . . # # . .
. # . . . . # . . # . . . . .
. . . . . . . . . # # . . . .
# . . . . . . . . . . . # # #
. . # . . . . . T . . # . # .
. . . . . . . . # . . . # # .
. . . # . . . . . . . . . # .
. # . . . . . . . . . . . . .
. . . . . . . . # # . . . . #
# # . . # . . . . . # # . . #
. . . . . . . . . . . . . P .
---------- [ BATTLE LOG ] ----------
 > Enemy defeated! +100 Score
 > Path blocked by wall!
 > Bought Spike Trap
 > Bought Spike Trap
 > Hand is full! Max 3 spells.
----------------------------------
 Spells: [1:Fireball (Direct)] [2:Spike Trap] [3:Spike Trap]
 [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис. 7 – Покупка трех заклинаний

7. Проверка использования заклинания прямого урона без врага.

```
 Spells: [1:Fireball (Direct)] [2:Spike Trap] [3:Spike Trap]
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command: 1
Direction (WASD) for Fireball (Direct): w
 [ PLAYER ] HP: 30/100 | Score: 100 | Mode: MELEE
 [ ENEMIES ]
  All enemies defeated!
----------------------------------------
. . . . . . . . . . . . # # .
. . . . . . . # . . . . . # .
. . . . . . . . . . . # . . . .
. . . . . # . # . . . # . . .
. . . # . . . . # . . # # . .
. # . . . . # . . # . . . . .
. . . . . . . . . # # . . . .
# . . . . . . . . . . . # # #
. . # . . . . . T . . # . # .
. . . . . . . . # . . . # # .
. . . # . . . . . . . . . # .
. # . . . . . . . . . . . . .
. . . . . . . . # # . . . . #
# # . . # . . . . . # # . . #
. . . . . . . . . . . . . P .
---------- [ BATTLE LOG ] ----------
 > Path blocked by wall!
 > Bought Spike Trap
 > Bought Spike Trap
 > Hand is full! Max 3 spells.
 > Casted Fireball (Direct)
----------------------------------
 Spells: [1:Fireball (Direct)] [2:Spike Trap] [3:Spike Trap]
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
```

Рис. 8 – Использования заклинания прямого урона без врагов

8. Проверка использования заклинания по площади без врага.

```
 Spells: [1:Blizzard (Area 2x2)]
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command: 1
Direction (WASD) for Blizzard (Area 2x2): w
 [ PLAYER ] HP: 100/100 | Score: 0 | Mode: MELEE
 [ ENEMIES ]
  E1: 30/30 HP [==========]
  E2: 30/30 HP [==========]
----------------------------------------
. . # . . # . # # . . # . . .
. . . . . . # . . . # . . . .
. P . . . . . # . . . . . . .
. . . . . . . # . . . . . . .
# # . . . . . # # . . . . . #
. . . E1. . . . . . . # . .
. . . . # . . . . . . . . . .
# . . . . . . . . . . . . . .
# # . . . . . . T . . . . # .
. . . . . . # . . . . . . . #
# . . . . . . # . . . . . . .
. . . . # . . . . . . . . . .
. # . . . . . . . . . # E2# . #
. . . . . . . . . . . . . . .
. . . . . . # . . . . . . . .
---------- [ BATTLE LOG ] ----------
 > Casted Blizzard (Area 2x2)
----------------------------------
 Spells:
  [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy [Q]-Quit
Enter command:
```

Рис. 9 – Использования заклинания по площади без врага

9. Нанесение урона от башни.

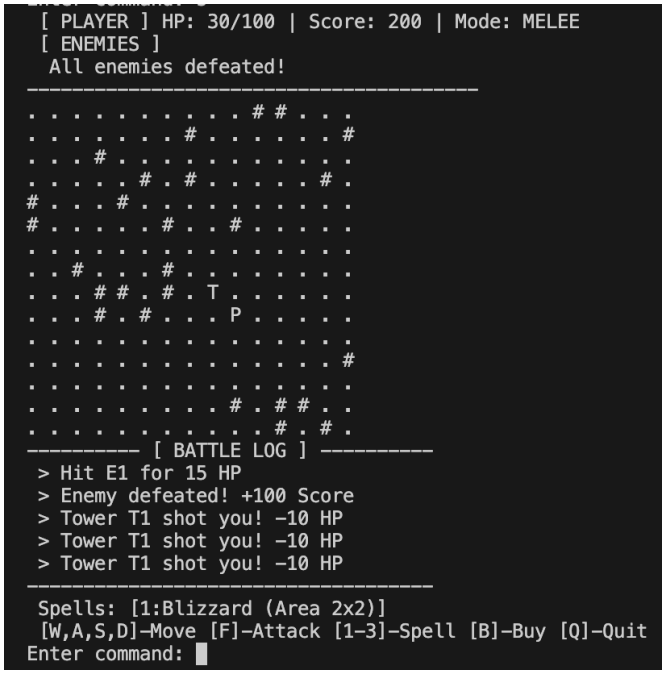

Рис. 10 – Нанесение урона от башни

Вывод

В ходе выполнения работы было дополнено игровое приложения на языке C++ с использованием объектно-ориентированного подхода. Были реализованы заклинания прямого урона, урона по площади и ловушки. Также были добавлены вражеские башни, которые атакуют игрока и перезаряжаются за несколько ходов. Во время тестирования приложения показало, что критических ошибок нет, все необходимые функции работают штатно.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: Makefile

```
CXX = g++
CXXFLAGS = -Wall -Wextra -std=c++17

TARGET = game

OBJS = main.o Game.o GameField.o Player.o Enemy.o GameCell.o Hand.o
\
          DirectDamageSpell.o      AreaDamageSpell.o      TrapSpell.o
EnemyTower.o

all: $(TARGET)

$(TARGET): $(OBJS)
     $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

main.o: main.cpp Game.h
     $(CXX) $(CXXFLAGS) -c main.cpp

Game.o: Game.cpp Game.h GameField.h Player.h Enums.h
     $(CXX) $(CXXFLAGS) -c Game.cpp

GameField.o: GameField.cpp GameField.h GameCell.h
     $(CXX) $(CXXFLAGS) -c GameField.cpp

Player.o: Player.cpp Player.h Enums.h
     $(CXX) $(CXXFLAGS) -c Player.cpp

Enemy.o: Enemy.cpp Enemy.h
     $(CXX) $(CXXFLAGS) -c Enemy.cpp

GameCell.o: GameCell.cpp GameCell.h Enums.h
     $(CXX) $(CXXFLAGS) -c GameCell.cpp

Hand.o: Hand.cpp Hand.h ISpell.h
     $(CXX) $(CXXFLAGS) -c Hand.cpp

DirectDamageSpell.o:   DirectDamageSpell.cpp   DirectDamageSpell.h
ISpell.h Enemy.h GameField.h
     $(CXX) $(CXXFLAGS) -c DirectDamageSpell.cpp

AreaDamageSpell.o:  AreaDamageSpell.cpp  AreaDamageSpell.h  ISpell.h
Enemy.h GameField.h
     $(CXX) $(CXXFLAGS) -c AreaDamageSpell.cpp

TrapSpell.o: TrapSpell.cpp TrapSpell.h ISpell.h GameField.h Enums.h
     $(CXX) $(CXXFLAGS) -c TrapSpell.cpp

EnemyTower.o: EnemyTower.cpp EnemyTower.h Player.h
     $(CXX) $(CXXFLAGS) -c EnemyTower.cpp

clean:
     rm -f $(OBJS) $(TARGET)
```

Название файла: ISpell.h

```
#ifndef ISPELL_H
#define ISPELL_H

#include <string>
#include <vector>

class GameField;
class Enemy;

class ISpell {
public:
    virtual ~ISpell() = default;
    virtual bool apply(int targetX, int targetY, GameField& field,
std::vector<Enemy>& enemies) = 0;
    virtual std::string getName() const = 0;
};

#endif
```

Название файла: AreaDamageSpell.h

```
#ifndef AREADAMAGESPELL_H
#define AREADAMAGESPELL_H

#include "ISpell.h"

class AreaDamageSpell : public ISpell {
public:
    bool apply(int targetX, int targetY, GameField& field,
std::vector<Enemy>& enemies) override;
    std::string getName() const override;
};

#endif
```

Название файла: AreaDamageSpell.cpp

```
#include "AreaDamageSpell.h"
#include "Enemy.h"
#include "GameField.h"

bool AreaDamageSpell::apply(int targetX, int targetY, GameField&
field, std::vector<Enemy>& enemies) {
    (void)field;
    for (auto& e : enemies) {
        if ((e.getX() == targetX || e.getX() == targetX + 1) &&
            (e.getY() == targetY || e.getY() == targetY + 1)) {
            e.takeDamage(30);
        }
    }
    return true;
}

std::string AreaDamageSpell::getName() const {
    return "Blizzard (Area 2x2)";
}
```

Название файла: DirectDamageSpell.h

```
#ifndef DIRECTDAMAGESPELL_H
#define DIRECTDAMAGESPELL_H

#include "ISpell.h"

class DirectDamageSpell : public ISpell {
public:
    bool apply(int targetX, int targetY, GameField& field,
std::vector<Enemy>& enemies) override;
    std::string getName() const override;
};

#endif
```

Название файла: DirectDamageSpell.cpp

```
#include "DirectDamageSpell.h"
#include "Enemy.h"
#include "GameField.h"

bool DirectDamageSpell::apply(int targetX, int targetY, GameField&
field, std::vector<Enemy>& enemies) {
    (void)field;
    for (auto& e : enemies) {
        if (e.getX() == targetX && e.getY() == targetY) {
            e.takeDamage(50);
            return true;
        }
    }
    return false;
}

std::string DirectDamageSpell::getName() const {
    return "Fireball (Direct)";
}
```

Название файла: TrapSpell.h

```
#ifndef TRAPSPELL_H
#define TRAPSPELL_H

#include "ISpell.h"

class TrapSpell : public ISpell {
public:
    bool apply(int targetX, int targetY, GameField& field,
std::vector<Enemy>& enemies) override;
    std::string getName() const override;
};

#endif
```

Название файла: TrapSpell.cpp

```
#include "TrapSpell.h"
#include "GameField.h"
```

```
#include "Enums.h"

bool TrapSpell::apply(int targetX, int targetY, GameField& field,
std::vector<Enemy>& enemies) {
    (void)enemies;
    if (targetX >= 0 && targetX < field.getWidth() && targetY >= 0
&& targetY < field.getHeight()) {
        if (field.getCell(targetX, targetY).getType() ==
CellType::EMPTY) {
            field.setCellType(targetX, targetY, CellType::TRAP);
            return true;
        }
    }
    return false;
}

std::string TrapSpell::getName() const {
    return "Spike Trap";
}
```

Название файла: EnemyTower.h

```
#ifndef ENEMYTOWER_H
#define ENEMYTOWER_H

class Player;

class EnemyTower {
private:
    int x, y;
    int damage;
    int range;
    int cooldown;
    int currentCooldown;

public:
    EnemyTower(int x, int y, int dmg, int rng, int cd);
    int update(Player& player);
    int getX() const { return x; }
    int getY() const { return y; }
};

#endif
```

Название файла: EnemyTower.cpp

```
#include "EnemyTower.h"
#include "Player.h"
#include <cmath>

EnemyTower::EnemyTower(int x, int y, int dmg, int rng, int cd)
    : x(x), y(y), damage(dmg), range(rng), cooldown(cd),
currentCooldown(0) {}

int EnemyTower::update(Player& player) {
    if (currentCooldown > 0) {
        currentCooldown--;
        return 0;
```

```
        }

        int px = player.getPosition().first;
        int py = player.getPosition().second;
        double dist = std::sqrt(std::pow(x - px, 2) + std::pow(y - py,
2));

        if (dist <= range) {
            player.takeDamage(damage);
            currentCooldown = cooldown;
            return damage;
        }
        return 0;
    }
```

Название файла: Hand.h

```
    #ifndef HAND_H
    #define HAND_H

    #include <vector>
    #include <memory>
    #include "ISpell.h"

    class Hand {
    private:
        std::vector<std::unique_ptr<ISpell>> spells;
        size_t maxSize;

    public:
        Hand(size_t size);
        bool addSpell(std::unique_ptr<ISpell> spell);
        void castSpell(size_t index, int x, int y, GameField& field,
std::vector<Enemy>& enemies);

        const std::vector<std::unique_ptr<ISpell>>& getSpells() const
{ return spells; }
    };

    #endif
```

Название файла: Hand.cpp

```
    #include "Hand.h"

    Hand::Hand(size_t size) : maxSize(size) {}

    bool Hand::addSpell(std::unique_ptr<ISpell> spell) {
        if (spells.size() < maxSize) {
            spells.push_back(std::move(spell));
            return true;
        }
        return false;
    }

    void Hand::castSpell(size_t index, int x, int y, GameField& field,
std::vector<Enemy>& enemies) {
        if (index < spells.size()) {
```

```
            if (spells[index]->apply(x, y, field, enemies)) {
                spells.erase(spells.begin() + index); // Удаляем п
осле использования
            }
        }
    }
```

Название файла: Player.h

```
#ifndef PLAYER_H
#define PLAYER_H

#include "Enums.h"
#include "Hand.h"
#include <utility>

class Player {
private:
    int hp;
    int damage;
    int score;
    std::pair<int, int> position;
    CombatMode mode;
    Hand hand;

public:
    Player(int startHp);

    int getHp() const;
    int getDamage() const;
    int getScore() const;
    std::pair<int, int> getPosition() const;
    CombatMode getMode() const;
    Hand& getHand();

    void takeDamage(int amount);
    void addScore(int points);
    void setPosition(int x, int y);
    void switchMode();
};

#endif
```

Название файла: Player.cpp

```
#include "Player.h"

Player::Player(int startHp)
    : hp(startHp), damage(15), score(0), mode(CombatMode::MELEE),
hand(3) {
    position = {1, 1};
}

int Player::getHp() const {
    return hp;
}

int Player::getDamage() const {
```

```
    return damage;
}

int Player::getScore() const {
    return score;
}

std::pair<int, int> Player::getPosition() const {
    return position;
}

CombatMode Player::getMode() const {
    return mode;
}

void Player::takeDamage(int amount) {
    hp -= amount;
    if (hp < 0) hp = 0;
}

void Player::addScore(int points) {
    score += points;
    if (score < 0) score = 0;
}

void Player::setPosition(int x, int y) {
    position = {x, y};
}

void Player::switchMode() {
    if (mode == CombatMode::MELEE) {
        mode = CombatMode::RANGED;
        damage = 5;
    } else {
        mode = CombatMode::MELEE;
        damage = 15;
    }
}

Hand& Player::getHand() {
    return hand;
}
```

Название файла: Game.h

```
#ifndef GAME_H
#define GAME_H

#include <vector>
#include <string>
#include "GameField.h"
#include "Player.h"
#include "Enemy.h"
#include "EnemyTower.h"
#include <iostream>

class Game {
private:
```

```
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    std::vector<std::string> battleLog;
    std::vector<EnemyTower> towers;
    bool isRunning;

    void addLog(const std::string& msg);
    void render();
    void handleInput();
    void update();

public:
    Game();
    void run();
};

#endif
```

Название файла: Game.cpp

```
#include "Game.h"
#include "Player.h"
#include "Enemy.h"
#include "GameField.h"
#include "GameCell.h"
#include "Enums.h"
#include "DirectDamageSpell.h"
#include "AreaDamageSpell.h"
#include "TrapSpell.h"
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <cmath>

Game::Game() : field(15, 15), player(100), isRunning(true) {
    std::srand(std::time(0));
    player.setPosition(1, 1);

    if (std::rand() % 2 == 0)

player.getHand().addSpell(std::make_unique<DirectDamageSpell>());
    else

player.getHand().addSpell(std::make_unique<AreaDamageSpell>());

    int tx = 8, ty = 8;
    field.setCellType(tx, ty, CellType::EMPTY);
    towers.push_back(EnemyTower(tx, ty, 10, 3 ,2));

    enemies.push_back(Enemy(30, 10, 5, 5));
    enemies.push_back(Enemy(30, 10, 12, 12));
}

void Game::render() {
    system("clear");

    // 1. Статус игрока
```

```
        std::cout << " [ PLAYER ] HP: " << player.getHp() << "/100 |
Score: " << player.getScore()
                  <<   "   |   Mode:   "   <<   (player.getMode()   ==
CombatMode::MELEE ? "MELEE" : "RANGED") << std::endl;

        // 2. Список врагов
        std::cout << " [ ENEMIES ] " << std::endl;
        if (enemies.empty()) std::cout << "  All enemies defeated!" <<
std::endl;
        for (size_t i = 0; i < enemies.size(); ++i) {
            int bars = (enemies[i].getHp() * 10) / enemies[i].getMaxHp();
            std::cout << "  E" << i+1 << ": " << enemies[i].getHp() <<
"/" << enemies[i].getMaxHp() << " HP [";
            for(int b=0; b<10; b++) std::cout << (b < bars ? "=" : " ");
            std::cout << "]" << std::endl;
        }
        std::cout   <<   "----------------------------------------"   <<
std::endl;

        // 3. Карта
        for (int y = 0; y < field.getHeight(); ++y) {
            for (int x = 0; x < field.getWidth(); ++x) {
                // ПРИОРИТЕТ ОТРИСОВКИ: Игрок > Враг >
Башня > Ловушка > Стена > Пустота

                // 1. Игрок
                if   (x   ==   player.getPosition().first   &&   y   ==
player.getPosition().second) {
                    std::cout << "P ";
                }
                else {
                    // 2. Проверка врагов
                    int enemyIdx = -1;
                    for (size_t i = 0; i < enemies.size(); ++i) {
                        if (enemies[i].getX() == x && enemies[i].getY()
== y) {
                            enemyIdx = i; break;
                        }
                    }

                    if (enemyIdx != -1) {
                        std::cout << "E" << enemyIdx + 1;
                    }
                    else {
                        // 3. Проверка башен
                        bool towerFound = false;
                        for (const auto& t : towers) {
                            if (t.getX() == x && t.getY() == y) {
                                std::cout << "T "; towerFound = true;
break;
                            }
                        }

                        if (!towerFound) {
                            // 4. Отрисовка элементов пол
я
```

```
                                CellType t = field.getCell(x, y).getType();
                                if (t == CellType::TRAP) std::cout << "^ ";
                                else if (t == CellType::OBSTACLE) std::cout
<< "# ";
                                else std::cout << ". ";
                            }
                        }
                    }
                }
                std::cout << std::endl;
            }

            // 4. Лог боя
            std::cout << "---------- [ BATTLE LOG ] ----------" << std::endl;
            for (const auto& log : battleLog) {
                std::cout << " > " << log << std::endl;
            }
            if(battleLog.empty()) std::cout << " > Waiting for action..." <<
std::endl;
            std::cout << "------------------------------------" << std::endl;

            // 5. Рука
            auto& spells = player.getHand().getSpells();
            std::cout << " Spells: ";
            for (size_t i = 0; i < spells.size(); ++i) std::cout << "[" <<
i+1 << ":" << spells[i]->getName() << "] ";
            std::cout << "\n [W,A,S,D]-Move [F]-Attack [1-3]-Spell [B]-Buy
[Q]-Quit" << std::endl;
        }

        void Game::handleInput() {
            char input;
            std::cout << "Enter command: ";
            std::cin >> input;

            bool turnTaken = false;

            if (input == 'q') {
                isRunning = false;
                return;
            }

            if (input == 'm') {
                player.switchMode();
                addLog("Switched to " + std::string(player.getMode() ==
CombatMode::MELEE ? "MELEE" : "RANGED"));
                turnTaken = true;
            } else if (input == 'w' || input == 's' || input == 'a' || input
== 'd') {
                int nextX = player.getPosition().first;
                int nextY = player.getPosition().second;

                if (input == 'w') nextY--;
                if (input == 's') nextY++;
                if (input == 'a') nextX--;
                if (input == 'd') nextX++;
```

```
            if (nextX >= 0 && nextX < field.getWidth() && nextY >= 0 &&
nextY < field.getHeight()) {
                if (!field.isObstacle(nextX, nextY)) {
                    player.setPosition(nextX, nextY);
                    turnTaken = true;
                } else {
                    addLog("Path blocked by wall!");
                }
            }
        }

        if (input == 'b') {
        if (player.getScore() >= 50) {
            std::unique_ptr<ISpell> newSpell;
            int r = std::rand() % 3;
            if (r == 0) newSpell = std::make_unique<DirectDamageSpell>();
            else      if      (r      ==      1)      newSpell      =
std::make_unique<AreaDamageSpell>();
            else newSpell = std::make_unique<TrapSpell>();

            if (player.getHand().addSpell(std::move(newSpell))) {
                player.addScore(-50);
                addLog("Bought                    "                    +
player.getHand().getSpells().back()->getName());
            } else {
                addLog("Hand is full! Max 3 spells.");
            }
        } else {
            addLog("Not enough points! (Need 50)");
        }
    }

        if (input >= '1' && input <= '3') {
            size_t spellIdx = static_cast<size_t>(input - '1');
            if (spellIdx < player.getHand().getSpells().size()) {
                std::cout    <<    "Direction    (WASD)    for    "    <<
player.getHand().getSpells()[spellIdx]->getName() << ": ";
                char dir; std::cin >> dir;
                int tx = player.getPosition().first;
                int ty = player.getPosition().second;
                if (dir == 'w') ty--; if (dir == 's') ty++; if (dir ==
'a') tx--; if (dir == 'd') tx++;

                addLog("Casted                    "                    +
player.getHand().getSpells()[spellIdx]->getName());
                player.getHand().castSpell(spellIdx,   tx,   ty,   field,
enemies);
                turnTaken = true;
            }
        }

        if (input == 'f') {
            int px = player.getPosition().first;
            int py = player.getPosition().second;
            bool hit = false;
            for (size_t i = 0; i < enemies.size(); ++i) {
                if   (std::abs(enemies[i].getX()   -   px)   <=   1   &&
std::abs(enemies[i].getY() - py) <= 1) {
```

```
                    int dmg = player.getDamage();
                    enemies[i].takeDamage(dmg);
                    addLog("Hit E" + std::to_string(i+1) + " for " +
std::to_string(dmg) + " HP");
                    hit = true;
                    turnTaken = true;
                    break;
                }
            }
            if (!hit) addLog("No enemies nearby!");
        }

        if (turnTaken) {
            for (auto& e : enemies) {
                e.moveTowardsPlayer(player, field);
            }
        }
    }

    void Game::update() {
        // 1. Атака башен
        for (size_t i = 0; i < towers.size(); ++i) {
            int dmgTaken = towers[i].update(player);
            if (dmgTaken > 0) {
                addLog("Tower T" + std::to_string(i + 1) + " shot you! -
" + std::to_string(dmgTaken) + " HP");
            }
        }

        // 2. Ловушки
        for (auto& e : enemies) {
            if    (field.getCell(e.getX(),    e.getY()).getType()    ==
CellType::TRAP) {
                e.takeDamage(40);
                field.setCellType(e.getX(), e.getY(), CellType::EMPTY);
                addLog("Enemy hit by TRAP! -40 HP");
            }
        }

        // 3. Смерть врагов и начисление очков
        for (auto it = enemies.begin(); it != enemies.end(); ) {
            if (it->getHp() <= 0) {
                player.addScore(100);
                addLog("Enemy defeated! +100 Score");
                it = enemies.erase(it);
            } else {
                ++it;
            }
        }

        // 4. Проверка смерти игрока
        if (player.getHp() <= 0) {
            isRunning = false;
        }
    }

    void Game::run() {
```

```
    while (isRunning) {
        render();
        handleInput();
        update();
    }
    render();
    std::cout << "\n!!! GAME OVER !!!" << std::endl;
}

void Game::addLog(const std::string& msg) {
    battleLog.push_back(msg);
    if (battleLog.size() > 5) {
        battleLog.erase(battleLog.begin());
    }
}
```